

SABER-PLR: 面向 hardest OOD 的 严重度与分支感知高效课程重放框架

Severity- and Branch-aware Efficient Replay Prioritization over Levels

文档目的 给出一套按工程现实逐步实现的新算法家族，用于在你们现有 PPO_NEW-v3/v3.1 主干之上，重构当前 PLR_UED 外层流程。设计目标有三点：

1. 保留现有 PLR/UED 在 $O_{\{10,90\}}$ hardest OOD 上对高严重度边界切片的有效修复能力；
2. 减少外层昂贵交互次数，并在完成 runtime profiling 后争取显著压低总耗时，而不是先验承诺固定的 wall-clock 降幅；
3. 在 telemetry 与特征导出链路打通的前提下，逐步让外层分布调度感知 v3 的三路结构 ($o, meta, \Delta o$)；若链路未打通，则 branch-aware 只能作为后续增强，而不是第一批必做项。

版本：v1.1

生成日期：2026 年 3 月 6 日

目录

1	问题重述与设计出发点	3
1.1	当前最关键的经验事实	3
1.2	当前必须承认的工程与证据边界	3
1.3	我们真正要修复的坏点	3
1.4	新算法总名称与版本路线	4
2	记号、主干网络与总体设计原则	4
2.1	内层 backbone: 保持 PPO_NEW-v3/v3.1 稳定主干不动	4
2.2	外层 level 定义	5
2.3	四条总原则	5
3	统一评分框架: 从 PLR 到 SABER-PLR	5
3.1	基础 PLR/UED 项	5
3.2	hardest-slice 定向项	6
3.3	可选研究扩展: branch-aware learnability	7
3.4	成本感知总评分	9
4	版本路线: 从简单到复杂逐步实现	9
4.1	V0: PLR-Lite (最小可替代版)	9
4.2	V1: SR-PLR (Severity-aware Removal PLR)	10
4.3	V2: SBR-PLR (Stream- and Branch-aware PLR)	11
4.4	V3: BSR-PLR (Budgeted Severity-aware Replay PLR)	12
4.5	V4: SABER-PLR (完整版)	14
5	统一训练阶段设计	15
5.1	Stage 0: inner backbone 预热	15
5.2	Stage 1: 候选生成	15
5.3	Stage 2: cheap probe 与评分	16
5.4	Stage 3: 选择与 full short-train	16
5.5	Stage 4: 回放更新与切换条件	16
6	推荐默认参数与调参顺序	16
6.1	推荐默认参数总表	16
6.2	建议的实现顺序	17
7	为什么 branch-aware 项值得加, 但不能当主角	17
7.1	值得加的理由	17

7.2	不能当主角的理由	18
8	实验建议、消融建议与预期结论口径	18
8.1	最关键的对比与消融	18
8.2	推荐主指标	18
8.3	应当如何写“理论安全性”	19
9	总结：建议你实际落地时如何推进	19

1 问题重述与设计出发点

1.1 当前最关键的经验事实

根据你们当前**已完成且可核对**的基线与 trace-level 切片分析，现阶段可以较稳地说：

1. 在 strongest OOD 分布 $O_{10,90}$ 上，普通 PPO 的平均回报长期停留在约 0.5 的低平台，这至少说明当前 hardest OOD 下存在稳定失效模式，而不只是单次随机波动；
2. PPO_NEW-v3/v3.1 作为 inner backbone 在 $F1$ 与部分 G 分布上有收益，但对 $O_{10,90}$ 并未稳定超过普通 PPO；
3. 当前 outer 系方法里，主结果上**明确跑赢 PPO** 的只有 **PLR_UED**；而且它的提升不是全局均匀抬升，而是集中修复了 **高严重度** $severity = 5/6 \times \text{removal}$ 的边界决策，并伴随少量 insertion 侧收益；
4. 因此当前证据更支持如下**优先级判断**，而不是绝对定理：

训练分布选择/调度效应 > inner 表征归纳偏置效应 > outer 控制器复杂度.

1.2 当前必须承认的工程与证据边界

为了避免后文把“研究设想”误写成“现成能力”，这里先把边界说清楚：

1. $O_{10,90}$ 上关于 hardest slice 的机制证据最强，但目前仍以有限 seeds 与有限完整 run 为主，因此适合指导算法设计，不宜写成过度确定的理论结论；
2. 当前最可信的 trace-level 信号是 $severity$ 、stream、语义动作、reward，以及 implement 路径上部分可用的 $p(a=1)$ ；训练路径并未稳定导出完整 $p(a=1)$ ；
3. 当前 PPO_NEW-v3 代码主干稳定暴露的是融合后的最终特征 h ，并未稳定导出三路 branch tensor 或 branch 级梯度，因此 branch-aware 设计目前只能视为**可选研究扩展**；
4. 当前 outer action space 本质上是离散 level 网格，而不是连续可微参数空间；因此任何“邻域编辑”都必须遵守当前 generator 真正支持的离散候选，而不是假设连续局部扰动天然存在。

1.3 我们真正要修复的坏点

记每个 implement 样本为

$$\tau_t = (s_t, a_t, r_t, sev_t, str_t),$$

其中 sev_t 表示严重度， $str_t \in \{\text{removal}, \text{insertion}\}$ 表示当前 stream。由已有切片结果可知，hardest OOD 的核心失败模式并非“所有样本都差”，而是集中在集合

$$\mathcal{H} = \{t \mid sev_t \geq \tau_H, str_t = \text{removal}\},$$

其中默认推荐 $\tau_H = 5$ 。

普通 PPO 在该集合上几乎完全塌缩到保守动作 $a=0$ ；而 PLR_UED 的收益则主要来自：

1. 在 $\mathcal{H}$ 上恢复少量但高质量的 $a = 1$;
2. 在 $sev = 5$ 左右保留更好的“继续保守也要保守得准确”的能力;
3. 不追求全局激进, 而追求 **selective action1 recovery**。

因此, 新算法不应再是“更通用的 outer RL”, 而应是:

围绕 **hardest slice** 进行定点课程调度的轻量级课程重放器。

1.4 新算法总名称与版本路线

本文给出一个从简单到复杂、便于逐版实现的算法家族, 统称

SABER-PLR: Severity- and Branch-aware Efficient Replay Prioritization over Levels

并拆为五个递进版本:

版本	名称	核心新增点
V0	PLR-Lite	最小可替代版: 保留 replay 与 learnability, 去掉重 outer RL, 仅做 score-based level selector。
V1	SR-PLR	加入 severity-aware + removal-aware 定向打分, 主攻 hardest slice。
V2	SBR-PLR	可选研究扩展 : 先做 V2a 的 proxy-only branch-aware, 再视 telemetry 情况升级到 V2b。
V3	BSR-PLR	在 V1 之上加入 learnability 硬门 + 两阶段预算控制与 cost-aware utility; 这是最有希望先落地降时的版本。
V4	SABER-PLR	完整增强版: 在 V1/V3 稳定后, 再考虑邻域编辑课程、adaptive new/replay mixing、轻量 selector 等机制。

2 记号、主干网络与总体设计原则

2.1 内层 backbone: 保持 PPO_NEW-v3/v3.1 稳定主干不动

我们不修改当前 PPO 主优化器, 只把 v3/v3.1 作为稳定 backbone。给定时刻 t 的输入, 定义

$$x_t = [o_t, b_t, a_{t-1}, r_{t-1}, \Delta o_t],$$

其中:

- o_t : 当前局部观测;
- b_t : 阶段位或 meta 状态;
- a_{t-1}, r_{t-1} : 最短闭环行为反馈;

- Δo_t : 短时变化量。

在 v3 中，这些信号被拆成三路：

$$z_t^{(o)} = f_o(o_t), \quad z_t^{(m)} = f_m(b_t, a_{t-1}, r_{t-1}), \quad z_t^{(\Delta)} = f_{\Delta}(\Delta o_t),$$

然后拼接或融合成

$$z_t = \phi(z_t^{(o)}, z_t^{(m)}, z_t^{(\Delta)}),$$

再经过 TemporalConv 或轻量时序聚合器得到最终策略特征 h_t ，最后进入 actor/critic 头。

本设计中，**inner PPO 训练逻辑不改**，新算法只改 outer 的分布选择、候选重放、评估预算与评分信号。

2.2 外层 level 定义

一个外层 level 记为

$$g = (\mu_A, \mu_B, p, n, pattern),$$

分别表示：

- μ_A, μ_B : 两类关键事件或上下文参数的均值；
- p : 混合比例或切换比例；
- n : 当前 generator 所支持的离散文件规模或离散长度档位；
- $pattern$: 当前 generator 所支持的离散模式。

在当前项目里，outer action space 更接近**离散 level 网格**，而不是连续可微参数控制。第一版实现应优先复用现有离散 key、现有 replay 机制与现有候选选择器，再讨论局部邻域编辑。

2.3 四条总原则

1. **定点修 hardest slice**。重点不是全局均值，而是 $severity \geq 5$ 的 removal 边界。
2. **选择性恢复 action1**。目标不是全局提高 $action1_rate$ ，而是只在“该出手”的高严重度边界上提高高置信度 $a = 1$ 。
3. **removal/insertion 非对称**。同一个二值动作在两个 stream 里的语义相反，因此 outer score 必须拆流统计。
4. **少算昂贵样本**。你们的瓶颈主要是 ALNS 相关外层交互成本，因此必须用两阶段预算与候选筛选减少 full evaluation 次数。

3 统一评分框架：从 PLR 到 SABER-PLR

3.1 基础 PLR/UED 项

对任意候选 level g ，记冻结策略或当前参考策略在该 level 上的表现为 $J_{frozen}(g)$ ，记短训或最新统计得到的表现为 $J(g)$ 。

沿用 PLR/UED 的核心思想，首先定义一个 **可学习门**：

$$L(g) = \exp\left(-\frac{(J_{frozen}(g) - c)^2}{2\sigma_L^2}\right). \quad (1)$$

当 $J_{frozen}(g)$ 太高时，说明太简单；太低时，说明太难。 $L(g)$ 迫使采样集中到“难但可学”的边界附近。

再定义学习进度项：

$$LP(g) = |J(g) - \bar{J}_{ref}(g)|, \quad (2)$$

其中 $\bar{J}_{ref}(g)$ 可以是上一轮、EMA 基线或历史缓冲统计。由于当前任务奖励近似二值，绝对回报对细粒度能力变化不够敏感，因此 $LP(g)$ 比单纯 $1 - J$ 更重要。

最后，沿用 replay buffer 的优先级更新：

$$q_g \leftarrow (1 - \beta_q)q_g + \beta_q \text{score}_g, \quad (3)$$

采样概率为

$$P(g) = \frac{(q_g + \varepsilon_q)^{\alpha_q}}{\sum_{g'} (q_{g'} + \varepsilon_q)^{\alpha_q}}. \quad (4)$$

3.2 hardest-slice 定向项

定义 hardest removal slice 与 hardest insertion slice 为

$$\mathcal{H}_{rem}(g) = \{t \mid \text{sev}_t \geq \tau_H, \text{str}_t = \text{removal}\}, \quad (5)$$

$$\mathcal{H}_{ins}(g) = \{t \mid \text{sev}_t \geq \tau_H, \text{str}_t = \text{insertion}\}. \quad (6)$$

默认 $\tau_H = 5$ 。为简化记号，后文若未特别说明， $\mathcal{H}(g) \equiv \mathcal{H}_{rem}(g)$ ，即主 **score** 默认只指向 **removal hardest slice**。同时定义 easy slice 为

$$\mathcal{E}(g) = \{t \mid \text{sev}_t \leq \tau_E\}, \quad \tau_E = 3. \quad (7)$$

当前证据表明，PLR_UED 的有效增益并不只是“多做 action1”，还包括 hardest slice 上继续保守时的正确性。因此至少需要五个定向统计量。

(1) hardest-slice 总体正确性

$$Q_{hard}(g) = \mathbb{E}_{t \in \mathcal{H}(g)}[r_t]. \quad (8)$$

它反映的是 hardest removal slice 的**总体可处理程度**。这一项必须保留，因为现有 trace 证据说明 PLR_UED 的收益不只来自恢复少量 $a = 1$ ，也来自高严重度下更准确的保守决策。

(2) 高严重度移除边界恢复分数

$$R_{hard}(g) = \mathbb{E}_{t \in \mathcal{H}(g)}[\mathbf{1}(a_t = 1) r_t]. \quad (9)$$

它度量：在 hardest slice 上，是否恢复了真正有用的 $a = 1$ 。但若单独使用这一项，容易把 score 推

向“过度追求 action1”，因此必须与 Q_{hard} 联合使用。

(3) 选择性置信分数 记策略在 removal 上预测 $a = 1$ 的概率为 $p_t = \pi_\theta(a = 1 | h_t)$ 。定义

$$C_{sel}(g) = \mathbb{E}_{t \in \mathcal{H}(g)} [p_t \mathbf{1}(a_t = 1) r_t] - \lambda_{fp} \mathbb{E}_{t \in \mathcal{E}(g)} [\mathbf{1}(a_t = 1)]. \quad (10)$$

第一项奖励“高严重度下高置信度的正确出手”，第二项惩罚 easy slice 上的无谓激进。

若当前 probe 链路无法稳定导出 p_t 或 $p(a = 1)$ ，则应退化为

$$\tilde{C}_{sel}(g) = \mathbb{E}_{t \in \mathcal{H}(g)} [\mathbf{1}(a_t = 1) r_t] - \lambda_{fp} \mathbb{E}_{t \in \mathcal{E}(g)} [\mathbf{1}(a_t = 1)], \quad (11)$$

或直接令 $w_c = 0$ 。这里不能把“理论上可算”误当成“当前工程上已稳定可算”。

(4) easy-slice 保护项

$$P_{easy}(g) = \mathbb{E}_{t \in \mathcal{E}(g)} [\mathbf{1}(a_t = 0) r_t]. \quad (12)$$

它用于抑制为了追求 hardest slice 而伤害 easy slice 的副作用。

(5) insertion 弱辅助监控项 由于 insertion 与 removal 语义相反，而且当前 trace 证据显示主收益集中在 removal hardest slice，因此 insertion 在 V1/V3 中不与 **removal** 对称建模。定义一个弱辅助监控量

$$M_{ins}(g) = \mathbb{E}_{t \in \mathcal{H}_{ins}(g)} [r_t]. \quad (13)$$

它只表示：若某些 candidate level 顺带改善了高严重度 insertion slice，可给予很小的辅助奖励。但第一版完全可以取 $w_i = 0$ ，把 M_{ins} 仅作为监控指标，而不进入主优化目标。这样可以明确避免把 insertion 与 removal 完全对称处理。

3.3 可选研究扩展：branch-aware learnability

这一节描述的是后续可选扩展，不是第一批实现的前置条件。当前代码与日志链路尚未稳定导出三路 branch tensor 与 branch 级梯度，因此这里更适合作为中后期研究方向，而不是首版工程规格。其目标不是强迫三路绝对均衡，而是：

避免长期只有一条分支承担 hardest slice 决策，而另外两条分支几乎失活或不可学习。

V2a: proxy-only branch-aware (当前更现实的首版) V2a 只依赖 forward-time 的 activation-level 或 output-level 信号，不要求 per-branch gradient telemetry。记 $u_t^{(b)} = \|z_t^{(b)}\|_2$ ，定义分支活性占比

$$a_{t,\text{act}}^{(b)} = \frac{u_t^{(b)}}{\sum_{b'} u_t^{(b')} + \varepsilon_a}, \quad b \in \{o, m, \Delta\}. \quad (14)$$

再定义一个轻量 ablation sensitivity。记 $\ell_t^{(-b)}$ 为将第 b 路分支置零、掩蔽或替换为固定基线后得到的当前动作 logit，则

$$\delta_t^{(b)} = |\ell_t - \ell_t^{(-b)}|, \quad \tilde{\delta}_t^{(b)} = \frac{\delta_t^{(b)}}{\sum_{b'} \delta_t^{(b')} + \varepsilon_\delta}. \quad (15)$$

在 hardest slice 上求均值得到

$$A_{b,\text{act}}(g) = \mathbb{E}_{t \in \mathcal{H}(g)} [a_{t,\text{act}}^{(b)}], \quad S_{b,\text{abl}}(g) = \mathbb{E}_{t \in \mathcal{H}(g)} [\tilde{\delta}_t^{(b)}]. \quad (16)$$

于是 V2a 的 proxy 分数定义为

$$S_{branch}^{V2a}(g) = \sum_{b \in \{o, m, \Delta\}} \log(\varepsilon_b + A_{b,\text{act}}(g) S_{b,\text{abl}}(g)) - \eta_b \text{Var}(A_{o,\text{act}} S_{o,\text{abl}}, A_{m,\text{act}} S_{m,\text{abl}}, A_{\Delta,\text{act}} S_{\Delta,\text{abl}}). \quad (17)$$

这一路线只需要前向可见的 **branch output** 与简易遮挡实验，因此是当前更现实的第一实现版本。

V2b: gradient-aware branch-aware (telemetry 打通后的增强版) 若后续 telemetry 打通，记 ℓ_t 为策略对当前动作的 logit。对每个 hardest slice 样本，定义三路分支的 **相对敏感度**：

$$s_t^{(b)} = \left\| \frac{\partial \ell_t}{\partial z_t^{(b)}} \right\|_2 \cdot \|z_t^{(b)}\|_2, \quad b \in \{o, m, \Delta\}. \quad (18)$$

归一化得到三路相对贡献：

$$a_t^{(b)} = \frac{s_t^{(b)}}{\sum_{b'} s_t^{(b')} + \varepsilon_a}. \quad (19)$$

在 hardest slice 上求均值得到

$$A_b(g) = \mathbb{E}_{t \in \mathcal{H}(g)} [a_t^{(b)}]. \quad (20)$$

仅有活性还不够，还需要“可学习”。因此再定义 probe 期间每条分支的梯度可学习度：

$$G_b(g) = \mathbb{E}_{t \in \mathcal{H}(g)} \left[\left\| \nabla_{\theta_b} \mathcal{L}_{ppo,t}^{\text{probe}} \right\|_2 \right], \quad (21)$$

并归一化为 $\tilde{G}_b(g) = G_b(g) / (\sum_{b'} G_{b'}(g) + \varepsilon_G)$ 。

最后构造更强的 gradient-aware 分数：

$$S_{branch}^{V2b}(g) = \sum_{b \in \{o, m, \Delta\}} \log(\varepsilon_b + A_b(g) \tilde{G}_b(g)) - \eta_b \text{Var}(A_o \tilde{G}_o, A_m \tilde{G}_m, A_\Delta \tilde{G}_\Delta). \quad (22)$$

其含义是：

- 每条分支都要在 hardest slice 上真正参与，且存在可学习梯度；
- 若只有一路过强、两路失活，则方差项给出惩罚；
- 若某条分支在当前候选 level 上几乎不起作用，则该 level 不应被过度优先。

在总分中统一记

$$S_{branch}(g) = \begin{cases} S_{branch}^{V2a}(g), & \text{若当前采用 V2a/proxy-only 模式,} \\ S_{branch}^{V2b}(g), & \text{若 telemetry 已打通且采用 V2b 模式.} \end{cases} \quad (23)$$

实现前提与保守落地方式 若要真正落地 branch-aware，建议按两级推进：

1. **V2a 先行**。只要求 backbone 在 forward 阶段能导出 $z^{(o)}, z^{(m)}, z^{(\Delta)}$ 或其等价 proxy，并允许做简易遮挡/置零实验；
2. **V2b 后上**。只有当 probe 路径允许对 branch 级变量反传并稳定记录梯度统计时，才启用 (18)–(22)；
3. 若连 V2a 的 activation/output proxy 都暂不可用，则直接把 w_b 置零，保持算法退化为 V1/V3。

为什么这是“软约束”而不是“硬均衡” 我们并不要求 $A_o = A_m = A_\Delta$ 。不同 slice 对三路的需求本就不同。我们只惩罚“单路塌缩”，不追求“完全平均”。因此 S_{branch} 只能作为辅助项，不能盖过 hardest-slice 主目标。

3.4 成本感知总评分

将以上各项合并，总评分写为

$$S_{raw}(g) = L(g) \left[w_q Q_{hard}(g) + w_h R_{hard}(g) + w_c C_{sel}(g) + w_e P_{easy}(g) + w_i M_{ins}(g) + w_p LP(g) + w_b S_{branch}(g) + w_n N(g) \right] - w_d D(g), \quad (24)$$

其中：

- $N(g) = 1/\sqrt{n_{seen}(g) + 1}$ 表示新颖度；
- $D(g)$ 表示文件生成失败、路径错误、空轨迹等工程惩罚；
- $w_q, w_h, w_c, w_e, w_i, w_p, w_b, w_n, w_d$ 为可调权重；
- 若 branch-aware 或 $p(a = 1)$ 统计暂不可用，则应显式把对应项关闭，而不是保留公式却在实现中隐式跳过。

这里推荐满足

$$w_q \geq w_h \gg w_i,$$

也就是：**hardest slice** 总体正确性至少不应比 **action1** 恢复更轻，而 **insertion** 只能作为弱辅助项或纯监控项。这样才能避免总分被“多做 action1”单独主导。

考虑训练成本后，真正用于 outer 选择的效用定义为

$$U(g) = \frac{S_{raw}(g)}{Cost(g) + \varepsilon_u}. \quad (25)$$

这一步非常关键，因为你们的外层瓶颈不是“网络更新慢”，而是“每个 outer 样本都贵”。

4 版本路线：从简单到复杂逐步实现

4.1 V0: PLR-Lite（最小可替代版）

目标

用最小风险替代当前高耗时 outer RL。V0 不加 branch-aware，也不加两阶段预算；只保留以下三件事：

1. learnability gate (1);
2. learning progress (2);
3. replay 更新 (3) 与优先级采样 (4)。

V0 的总分数定义为

$$S_{V0}(g) = L(g)[w_{\Delta}LP(g) + w_JJ(g) + w_nN(g)] - w_dD(g). \quad (26)$$

适用场景

- 先替换 EDRL/RARL 这类更重 outer 控制器；
- 验证“轻量 score-based selector + replay”就能跑通；
- 作为后续所有版本的保底退化路径。

推荐参数

参数	推荐值	说明
c	0.55	learnability 中心, 落在“过易与过难之间”的中间偏低位置
σ_L	0.18	可学习带宽
α_q	0.9	priority exponent
β_q	0.20	score EMA
ε_q	1e-4	防止零概率
$w_{\Delta}, w_J, w_n, w_d$	0.45, 0.25, 0.15, 0.15	先偏向 learning progress
replay/new	0.65 / 0.35	初始混合
buffer size	64	小而稳, 便于快速周转

4.2 V1: SR-PLR (Severity-aware Removal PLR)

目标

从“通用 high-value replay”升级为“定向 hardest slice replay”。V1 的关键不是简单追求更多 $a = 1$, 而是把 score 从全局统计改成针对 hardest removal slice 的边界决策质量统计。

总分数

$$S_{V1}(g) = L(g)[w_qQ_{hard}(g) + w_hR_{hard}(g) + w_cC_{sel}(g) + w_eP_{easy}(g) + w_iM_{ins}(g) + w_pLP(g) + w_nN(g)] - w_dD(g). \quad (27)$$

核心变化

1. 用 $Q_{hard} + R_{hard}$ 取代通用 **difficulty**。优先修复 $sev \geq 5$ 的 removal 边界，而不是全局最难分布；
2. 引入 C_{sel} ，但允许回退。若 $p(a=1)$ 暂不可稳定导出，则使用 $\tilde{C}_{sel}$ 或直接关闭该项；
3. 引入 P_{easy} 。保住 easy slice 的校准与稳定性，避免为了 hardest slice 而整体变得激进；
4. **insertion** 只做弱辅助。 M_{ins} 只占很小权重，第一版甚至可以设为 0，仅保留监控。

推荐参数

参数	推荐值	说明
τ_H	5	hard severity threshold
τ_E	3	easy threshold
λ_{fp}	0.20	easy slice 假阳性惩罚
$w_q, w_h, w_c, w_e, w_i, w_p, w_n, w_d$	0.28, 0.18, 0.14, 0.15, 0.05, 0.10, 0.05, 0.05	让 hardest 总体正确性重于 action1 恢复, insertion 仅弱辅助
$w_q : w_h : w_i$	0.28 : 0.18 : 0.05	满足 $w_q \geq w_h \gg w_i$ ，避免主目标被“多做 action1”带偏

4.3 V2: SBR-PLR (Stream- and Branch-aware PLR)

目标

V2 解决“outer 调度与 v3 主干脱节”的问题，但前提是 branch telemetry 至少部分可见。除保留 V1 所有项外，新增 $S_{branch}(g)$ 作为辅助项。这里进一步分成两个现实层级：

1. **V2a: proxy-only branch-aware**。只依赖 activation-level / output-level proxy，是当前更现实的第一实现版本；
2. **V2b: gradient-aware branch-aware**。只有在 telemetry 打通后才启用，是后续增强版。

若连 V2a 的 proxy 都暂不可稳定导出，则 V2 应继续关闭，不应强行上线。

总分数

$$S_{V2}(g) = S_{V1}(g) + L(g) w_b S_{branch}(g). \quad (28)$$

关键思想

- 如果某个 level 只能把 $z^{(o)}$ 打热，而 meta 与 Δo 路分支几乎无活性或无敏感度，则它更像“单路刷分 level”，不应被过度优先；
- 如果某个 level 在 hardest slice 上使三路都产生非零可学习信号，说明它更能训练出稳健边界；

- 这样外层选择的不只是“短期提分 level”，而是“更能训练 v3 结构先验的 level”；
- 但这条思路只有在 branch proxy 与真实性能确实相关时才成立，因此 V2 必须默认带退化开关；
- 工程上应先上 V2a，再决定是否值得升级到 V2b。

推荐参数

参数	推荐值	说明
w_b	0.05 – 0.10	只做辅助项；V2a 建议从 0.05 起，V2b 再视效果上调
η_b	0.35	分支失衡惩罚
$\varepsilon_a, \varepsilon_\delta, \varepsilon_G, \varepsilon_b$	1e-6	数值稳定
V2a 统计样本	hardest slice only	只在 $sev \geq 5$, removal 上统计 branch proxy
V2b 梯度采样频率	每个 probe 末 1 次	降低额外开销；若无法稳定反传，则停留在 V2a

保守开关机制 为避免 S_{branch} 误导总分数，推荐加入保守门控：若最近 m 轮验证中， S_{branch} 与 ΔJ 的秩相关系数不为正，则暂时将 w_b 退火到 0。即

$$w_b(t+1) = \begin{cases} \rho_w w_b(t), & \text{if } \text{Corr}(S_{branch}, \Delta J) \leq 0, \\ \min(w_b^{max}, w_b(t)/\rho_w), & \text{otherwise,} \end{cases} \quad (29)$$

其中 $\rho_w = 0.5$ 推荐。这样即便 branch-aware 信号暂时无效，也能自动退化回 V1。

4.4 V3: BSR-PLR (Budgeted Severity-aware Replay PLR)

目标

V3 是最有希望明显降时的关键版本。它加入两阶段预算控制，把大部分无效 full evaluation 换成廉价 probe。这里的“更快”首先指外层 candidate-evaluation 成本更低，而不是先验承诺整个实验 wall-clock 必然按同一比例下降。

两阶段流程

阶段 A: cheap probe 对每个候选 level 只做少量 rollout 或极短 inner short-train，得到

$$J_{frozen}(g), LP(g), Q_{hard}(g), R_{hard}(g), C_{sel}(g), P_{easy}(g), M_{ins}(g), S_{branch}(g), D(g).$$

基于这些量计算 $U(g)$ 。

阶段 B: full short-train 只对满足 $L(g) \geq \tau_L$ 的候选中、 $U(g)$ 前 M 个 level 做正式短训，并用训练后的真实结果更新 q_g 。

预算函数

为避免把大预算浪费在“太差但其实不可学”的候选上，先定义 learnability 硬门

$$\Gamma_L(g) = \mathbf{1}(L(g) \geq \tau_L). \quad (30)$$

记 full short-train 预算为 $B(g)$ ，定义

$$B(g) = \begin{cases} 0, & L(g) < \tau_L, \\ B_{min} + (B_{max} - B_{min}) \cdot \text{sigmoid}(\alpha_B(\tau_U - U(g)) + \beta_B(\tau_R - R_{hard}(g))), & L(g) \geq \tau_L. \end{cases} \quad (31)$$

解释如下：

- 若 $L(g) < \tau_L$ ，则该候选**直接跳过 full evaluation**，因为它已经落到“太难或不可学”的一侧；
- 若 $U(g)$ 高、 $R_{hard}(g)$ 也高，说明该 level 已经学得不错或 probe 已充分，预算靠近 B_{min} ；
- 若 $U(g)$ 尚可但 $R_{hard}(g)$ 仍差，说明这是“仍值得进一步修 hardest slice”的 level，预算靠近 B_{max} 。

成本感知效用

V3 采用 (25)，其中

$$Cost(g) = c_0 + c_1 T_{probe}(g) + c_2 B(g), \quad (32)$$

$T_{probe}(g)$ 为 probe 时间或统一设定的 probe 步数。

推荐参数

参数	推荐值	说明
候选数 N_{cand}	6	每轮只看 6 个候选，控制外层成本
full 进入数 M	2	只对前 2 个做正式短训
B_{min}, B_{max}	0.25, 1.00	相对当前 full short-train 标准预算的比例
probe 预算	0.12 – 0.18 倍 full	一般推荐 0.15 倍
τ_L	0.30 – 0.40	learnability 硬门，建议先从 0.35 起步
α_B, β_B	3.0, 4.0	让 hardest-recovery 主导预算分配
τ_U, τ_R	0.55, 0.20	utility 与 hard-recovery 的中间门槛

理论上的加速来源 若当前 PLR_UED 每轮对 N_{cand} 个候选都做 full evaluation，则其每轮成本近似为

$$T_{old} \approx N_{cand} C_{full}.$$

V3 后变为

$$T_{new} \approx N_{cand} C_{probe} + M C_{full},$$

若取 $N_{cand} = 6$, $M = 2$, $C_{probe} = 0.15C_{full}$, 则

$$\frac{T_{new}}{T_{old}} \approx \frac{6 \times 0.15 + 2}{6} = 0.4833.$$

这说明: 若只看候选评估型训练开销, V3 在结构上有机会把该部分压到旧版本的约 48%。但这不等于最终 wall-clock 必然减半, 因为真实总耗时还包含事件文件生成、子进程启动、日志落盘、checkpoint 切换等额外成本。因此 V3 的速度收益必须配合 runtime profiling 报告, 不能只靠公式口径下结论。

4.5 V4: SABER-PLR (完整版)

目标

V4 在 V3 的基础上, 把“找对 level”这件事做得更稳。它新增三项机制:

1. 邻域编辑课程: 从 top replay level 出发做局部扰动, 不再只重放单点;
2. adaptive new/replay mixing: 根据当前 hardest slice 修复情况调节探索比例;
3. 轻量 contextual bandit/UCB 选择器: 优先复用或轻改现有 UCB/TS 式候选选择器, 而不是再引入一套重 outer RL。

邻域编辑

从 replay top- K level 中抽一个种子 g_i , 做局部编辑:

$$g' = (\mu_A + \delta_A, \mu_B + \delta_B, p + \delta_p, n + \delta_n, pattern'), \quad (33)$$

其中推荐默认扰动集合:

$$\delta_A, \delta_B \in \{\pm 10\}, \quad \delta_p \in \{\pm 0.1\}, \quad \delta_n \in \mathcal{N}_{files}^{adj}(n).$$

其中 $\mathcal{N}_{files}^{adj}(n)$ 表示当前 generator 所支持的离散相邻文件规模档位, 而不是默认存在连续的 ± 100 扰动。对 $pattern$ 同理: 第一版只能在当前 generator 真正支持的模式集合内切换; 若当前实现只稳定支持少数模式, 则不得把 $ab \leftrightarrow aba \leftrightarrow abba$ 这类编辑写成现成能力。

adaptive new/replay mixing

定义最近窗口内 hardest-slice 修复率为

$$\rho_{hard}(t) = \frac{1}{W} \sum_{k=t-W+1}^t \mathbb{E}[\mathbf{1}(sev \geq 5, str = \text{removal}, a = 1) r]. \quad (34)$$

再定义当前新样本概率:

$$p_{new}(t) = \text{clip} \left(p_0 + k_1(H^* - \bar{H}_t) + k_2(\rho_{hard}^* - \rho_{hard}(t)), p_{min}, p_{max} \right), \quad (35)$$

其中 $\bar{H}_t$ 可为 recent entropy 或 recent diversity 指标。含义是:

- 若动作熵过低、hardest recovery 仍弱，则提高 p_{new} 去找新 level;
- 若 hardest recovery 已起色但不稳定，则提高 replay 比例做巩固训练。

轻量选择器：UCB over utility

不训练重 outer RL，只在当前候选集上做轻量 UCB：

$$A(g) = U(g) + \beta_{ucb} \sqrt{\frac{\log(t+1)}{n(g)+1}}, \quad (36)$$

每轮取 $\arg\max_g A(g)$ 作为进入 full short-train 的主候选之一。

推荐参数

参数	推荐值	说明
replay top- K	8	从 top-8 中采样邻域种子
邻域编辑比例	0.5	新候选中一半来自邻域编辑
p_0, p_{min}, p_{max}	0.35, 0.15, 0.65	新样本基础概率
k_1, k_2	0.20, 0.25	熵与 hardest-recovery 双调节
β_{ucb}	0.05 – 0.12	轻量探索系数
窗口 W	10	统计 $\rho_{hard}(t)$ 的滑窗长度

5 统一训练阶段设计

5.1 Stage 0: inner backbone 预热

保持你们现有的 Phase1，不做任何变动：

1. 用当前 base 分布训练出稳定的 θ_{base} ;
2. 缓存基础统计：整体 reward、action ratio、trace 切片、hardest slice 基线;
3. 初始化 replay buffer，可以把 base 附近的一小部分分布作为“热启动候选”。

5.2 Stage 1: 候选生成

每一轮 outer 先构造候选集 $\mathcal{C}_t$ ，由三部分组成：

$$\mathcal{C}_t = \mathcal{C}_t^{replay} \cup \mathcal{C}_t^{edit} \cup \mathcal{C}_t^{new}. \quad (37)$$

其中：

- $\mathcal{C}_t^{replay}$ ：从优先级 buffer 中抽的历史高价值 level;
- $\mathcal{C}_t^{edit}$ ：从 replay top level 生成的邻域编辑 level;
- $\mathcal{C}_t^{new}$ ：按当前 $p_{new}(t)$ 抽的全新 level。

5.3 Stage 2: cheap probe 与评分

实现前提 在进入这一阶段之前，至少要保证 probe/trace 链路能稳定导出 *severity*、stream、语义动作、reward；若 $p(a=1)$ 或 branch 统计暂不可用，则对应项必须显式关闭，而不是在文档里保留、在实现里默默跳过。

对 $\mathcal{C}_t$ 中每个候选 level:

1. 生成对应事件文件；
2. 用冻结或当前 backbone 做极短 rollout / short-train；
3. 只在 trace 中抽取 hardest 与 easy slice；
4. 计算 $L, LP, Q_{hard}, R_{hard}, C_{sel}, P_{easy}, M_{ins}, S_{branch}, D$ ；
5. 算出 $S_{raw}(g)$ 与 $U(g)$ 。

5.4 Stage 3: 选择与 full short-train

1. 先按 $\Gamma_L(g) = \mathbf{1}(L(g) \geq \tau_L)$ 过滤掉不可学候选，再用 (36) 或直接按 $U(g)$ 排序，选出前 M 个候选；
2. 对每个被选中的候选，根据 (31) 分配预算 $B(g)$ ；
3. 在该候选上做正式 short-train，得到真实 $J(g)$ 与 trace 统计；
4. 用真实结果更新 q_g 、 $n_{seen}(g)$ 、 $\rho_{hard}(t)$ 、 $p_{new}(t)$ 。

5.5 Stage 4: 回放更新与切换条件

若最近 W 轮满足以下任一条件，可进入下一阶段或触发终止：

1. $\rho_{hard}(t)$ 持续上升且方差收缩；
2. $R_{hard}(g)$ 进入平台区，但 easy slice 未明显受损；
3. replay top level 已出现稳定的邻域簇，而非单点垄断；
4. 达到外层预算上限。

6 推荐默认参数与调参顺序

6.1 推荐默认参数总表

类别	参数	推荐值
learnability	c, σ_L	0.55, 0.18
priority replay	$\alpha_q, \beta_q, \epsilon_q$	0.9, 0.20, 10^{-4}
hard/easy slice	τ_H, τ_E	5, 3
selective action	λ_{fp}	0.20

类别	参数	推荐值
stream asymmetry	w_i / monitor	$w_i = 0.05$ 或直接取 0, 仅把 insertion 作为弱辅助或监控项
score weights (V1)	$w_q, w_h, w_c, w_e,$	0.28, 0.18, 0.14, 0.15
	w_i, w_p, w_n, w_d	0.05, 0.10, 0.05, 0.05
branch-aware	w_b, η_b	0.05–0.10, 0.35; V2a 先上, V2b 后上
probe/full	N_{cand}, M	6, 2
budget	B_{min}, B_{max}, τ_L	0.25, 1.00, 0.35
probe ratio	C_{probe}/C_{full}	0.15
budget slope	α_B, β_B	3.0, 4.0
new/replay	p_0, p_{min}, p_{max}	0.35, 0.15, 0.65
adaptive mix	k_1, k_2	0.20, 0.25
UCB	β_{ucb}	0.08
edit neighborhood	$\delta_\mu, \delta_p, \delta_n$	$\pm 10, \pm 0.1, \mathcal{N}_{files}^{adj}(n)$
window	W	10

6.2 建议的实现顺序

为降低工程风险，建议按下面顺序推进，而不是一次性上完整版：

1. 先做 **V0**。目标是用最小替换跑通 replay + score-selector 的主链路；
2. 再做 **V1**。一旦 trace 级 hardest slice 指标能稳定导出，就立刻切到定向 score；
3. 再做 **V3**。因为你当前最痛的是时间，预算控制应早于 branch-aware 上线；
4. 然后再决定是否做 **V2**。若 branch proxy 已可稳定导出，先做 V2a；只有当 V2a 确实有效且 telemetry 打通后，再考虑 V2b；
5. 最后做 **V4**。邻域编辑与 adaptive mixing 放到稳定主链之后，不要一开始全上。

如果只能做三版，我建议依次做：

$$V0 \rightarrow V1 \rightarrow V3.$$

因为这三版已经覆盖：**替换重 outer**、**定点修 hardest slice**、**显著降时**这三件核心事情。

7 为什么 branch-aware 项值得加，但不能当主角

7.1 值得加的理由

若没有 $S_{branch}(g)$ ，新算法仍然可能有效，但它更像“PLR_UED 外层 + v3 内层”的并列组合。加入 $S_{branch}(g)$ 后，outer 在挑分布时会回答一个更细的问题：

哪些 level 不仅能提分，而且在 hardest slice 上更可能训练出三路可学习而非单路塌缩的表征？

这样就把 v3 的结构先验抬升成了 outer 的候选选择依据。但必须强调：这是一种**研究扩展价值**，不是当前项目里已经稳定上线的能力。工程上更合理的做法是先验证 V2a 的 proxy-only 版本，再决定是否值得走到 V2b。

7.2 不能当主角的理由

当前已有证据说明，hardest OOD 的主导因素仍是 **训练分布调度**，而不是继续加重 inner 表征。因此：

- S_{branch} 应作为辅助项，权重不宜过大；
- 一旦发现 S_{branch} 与真实提升脱钩，就应通过 (29) 自动退化；
- 即使最终发现 branch-aware 项收益有限，V2 也应稳定退化回 V1，V3 仍可独立作为速度主线存在。

8 实验建议、消融建议与预期结论口径

8.1 最关键的对比与消融

推荐最少做以下对比：

1. PPO vs PPO_NEW-v3/v3.1 vs PLR_UED vs V1 vs V3；
2. 在 $O_{10,90}$ 上至少做 3 seeds，并强制报告 trace-level slice，而不是只报平均回报；
3. V1 去掉 Q_{hard} （验证 hardest-slice 总体正确性是否必要）；
4. V1 去掉 C_{sel} （验证 selective action1 是否必要）；
5. V1 令 $w_i = 0$ （验证 insertion 弱辅助是否真的有价值）；
6. V1 去掉 P_{easy} （验证 easy-slice 保护是否必要）；
7. V2a vs V2b（验证 proxy-only 版本是否已经足够）；
8. V2 去掉 S_{branch} （验证 branch-aware 是否真的增益）；
9. V3 去掉 learnability gate 或去掉两阶段预算（验证主要加速来源）；
10. V4 去掉邻域编辑（验证是否避免只记单点）。

8.2 推荐主指标

不建议只报 average reward。至少要同时报告：

- hardest slice overall quality: Q_{hard} ；
- hardest slice recovery: R_{hard} ；
- selective action quality: C_{sel} ；
- easy-slice preservation: P_{easy} ；

- insertion weak monitor: M_{ins} ;
- 总体平均回报;
- 外层 wall-clock、单轮候选评估耗时以及 $\Gamma_L(g) = 0$ 的跳过比例;
- replay 覆盖率、top-k 邻域覆盖率。

8.3 应当如何写“理论安全性”

不能诚实地声称“真实测试回报必定提升”，因为这需要超出当前信息的严格证明。但可以较稳地表述为：

1. 新算法是对现有 PLR 主线的保守增强，而非推翻；
2. 若新增 hardest-slice 或 branch-aware 信号无效，权重可自动退化到 0，算法退化为 V0/V1；
3. 两阶段预算控制对外层候选评估型训练开销有明显的削减机制，因此“更快”可以给出结构性解释；但这不是对最终 wall-clock 减半的直接保证；
4. hardest-slice 定向 score 是对当前 trace 证据的直接利用，因此比通用外层难度目标更贴近真实失效模式。

9 总结：建议你实际落地时如何推进

一句话总结

不要再把主要创新放在更复杂的 outer RL 控制器上，而要把现有 PLR/UED 提炼成一个“定点修复 severity 5/6 removal 边界、以 V1/V3 为主线、先做可验证的速度优化、branch-aware 仅作后续可选增强”的高样本效率课程重放框架。

更具体地说：

1. 第一批实现建议先做 V0，快速替掉重 outer RL；
2. 第二批直接切到 V1，把 score 对齐到 hardest slice；
3. 若时间压力很大，第三批优先做 V3，而不是先做 V2；
4. branch-aware 项值得做，但应视为“在 telemetry 打通后再验证”的辅助增强，而不是第一优先级；
5. 完整版 V4 适合作为论文中的主方法，但工程实现上应在 V1/V3 验证稳定后再上。

推荐命名

工程主线名：SR-PLR / BSR-PLR

中文：严重度感知的高效优先级课程重放

若后续 branch-aware 扩展被真实验证，再把完整版命名为：SABER-PLR。这样命名更符合“先落地 V1/V3，再决定是否引入 V2/V4”的实现顺序。